

AI-Powered Behavioral Anomaly Detection for Cybersecurity

Technical Report

Dataset: 108,986 sessions across 330 entities, 2.006% injected anomaly rate
Alert queue: 1,971 alerts generated from the full dataset
Train/test split: 76,290 train / 32,696 test rows (chronological split at 2026-06-10 16:17:15)

1. Executive Summary

The system detects behavioral anomalies in access logs using an unsupervised Isolation Forest detector (supplemented with a targeted rule for device-fingerprint mismatches), classifies flagged events into one of 7 attack categories with a Random Forest, and explains each alert using both statistical deviation analysis and exact SHAP-based feature attribution.

On a held-out chronological test split, the detector achieves **99.7% precision** and **58.2% recall** (ROC-AUC 0.974, PR-AUC 0.826), with a false positive rate of **0.0%**. At a realistic 1% analyst alert budget, precision reaches **99.7%**, capturing **59.3%** of all true anomalies.

Across the full dataset’s alert queue, **1,554 of 2,186** (71.1%) true attack sessions were actually flagged and are visible to an analyst - see Section 5 for the important caveat that this varies sharply by attack type.

2. Dataset

Synthetic access-log data generated per the required schema, with all 7 specified attack patterns injected at controlled rates plus an 8th ambiguous edge case (insider drift) for false-positive tuning. Full behavioral assumptions and the attack taxonomy are documented in `generator/ASSUMPTIONS.md`.

Label	Count
normal	106,800
brute force	1,163
credential stuffing	379
low and slow exfiltration	308
lateral movement	177
insider drift	88
impossible travel	46
device spoofing	25

3. System Architecture

Synthetic Data Generator (Phase 1)
|

ML Pipeline (Phase 3)

Baseline Profiler (rolling per-entity window, learned from logs)

-> Feature Engineering (15 signals)

-> Isolation Forest Detector (unsupervised)

-> Random Forest Classifier (attack-type)

Alert Population (Phase 4) -- hybrid statistical + rule-based flagging

Explainability (Phase 5) -- z-score reasoning + SHAP classification attribution

Analyst Dashboard (Phase 4) -- Alert Queue, Live Feed, Entity Explorer,

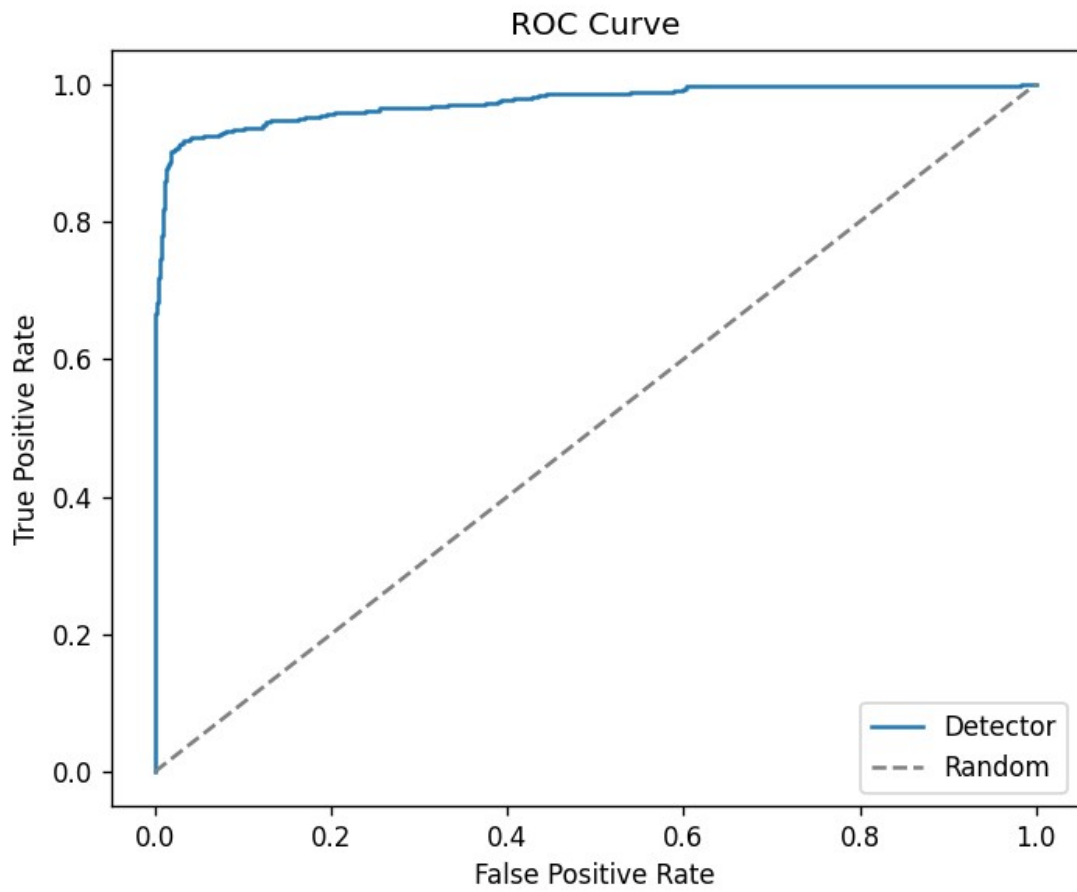
Device Trust. World Map. Model

Evaluation

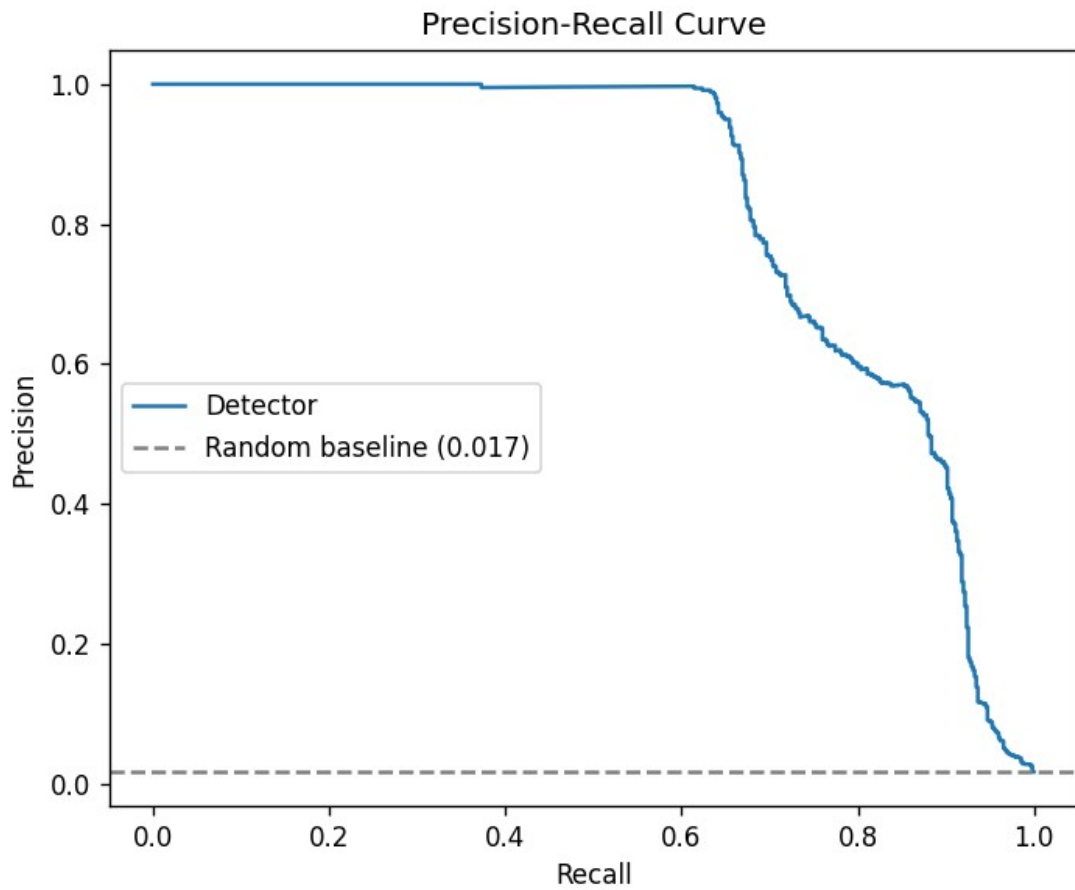
4. Model Performance (Held-Out Test Split)

4.1 Binary Detection

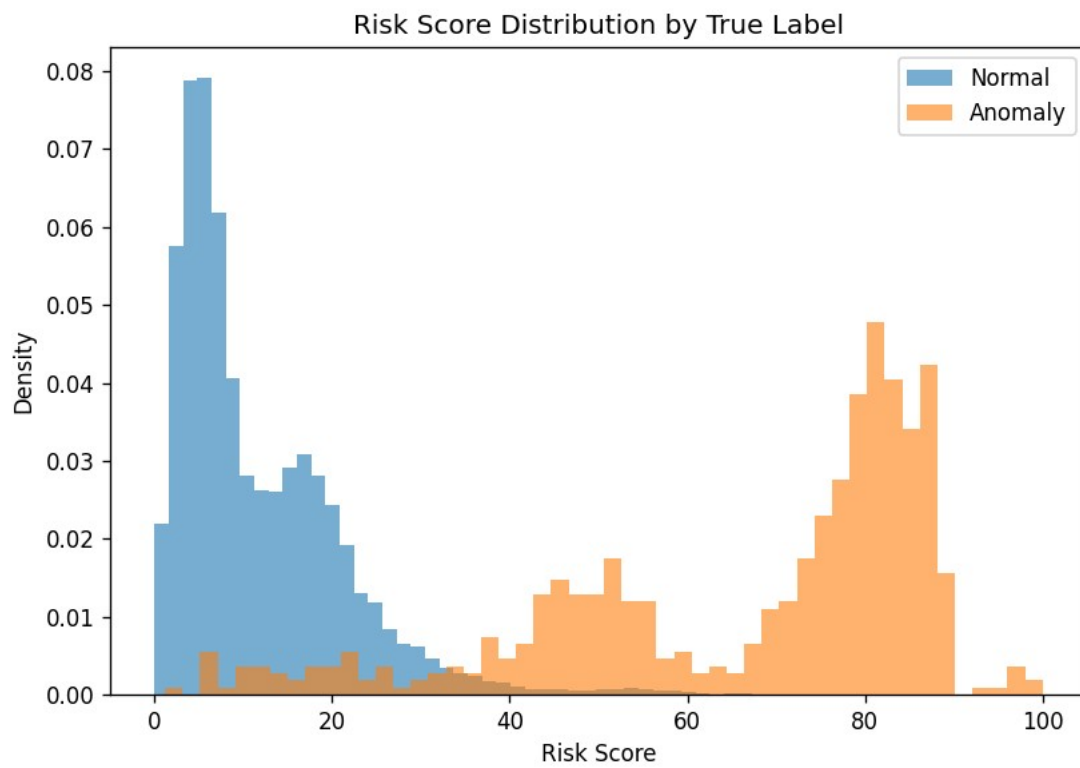
Metric	Value
Precision	0.997
Recall	0.582
F1	0.735
ROC-AUC	0.974
PR-AUC	0.826
False Positive Rate	0.0%
Precision @ top 1pct alert budget	0.997 (captures 59.3% of anomalies)
Precision @ top 2pct alert budget	0.639 (captures 76.0% of anomalies)
Precision @ top 5pct alert budget	0.309 (captures 91.8% of anomalies)



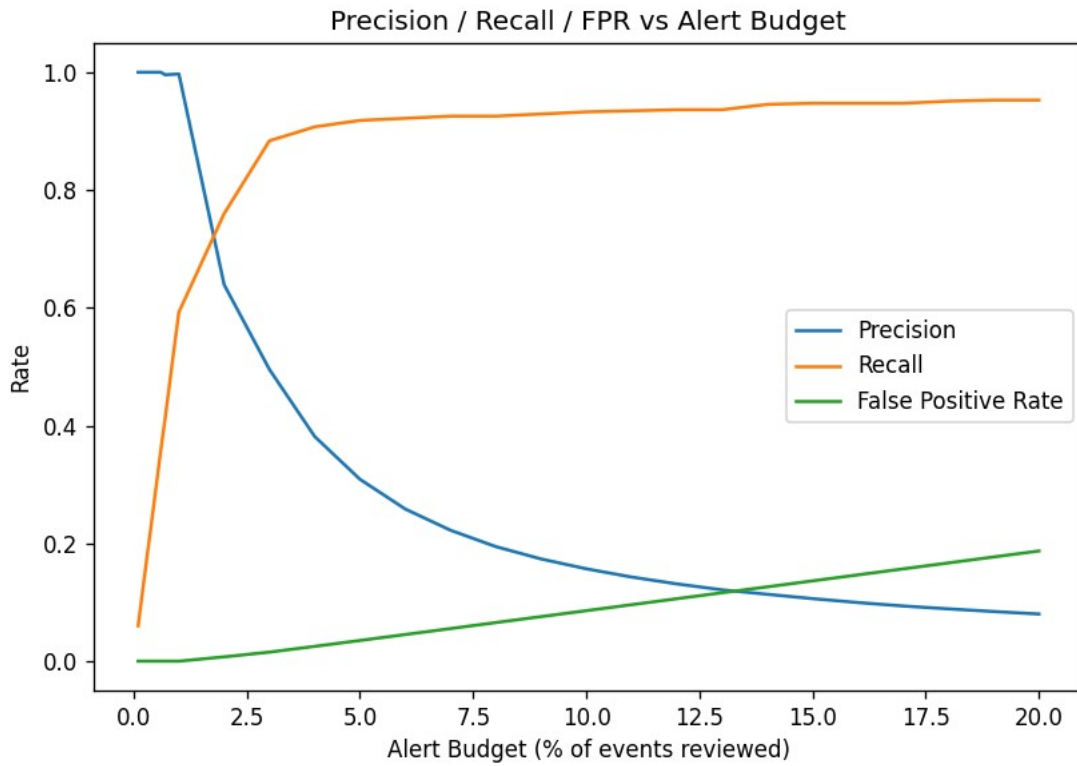
ROC Curve



Precision-Recall Curve



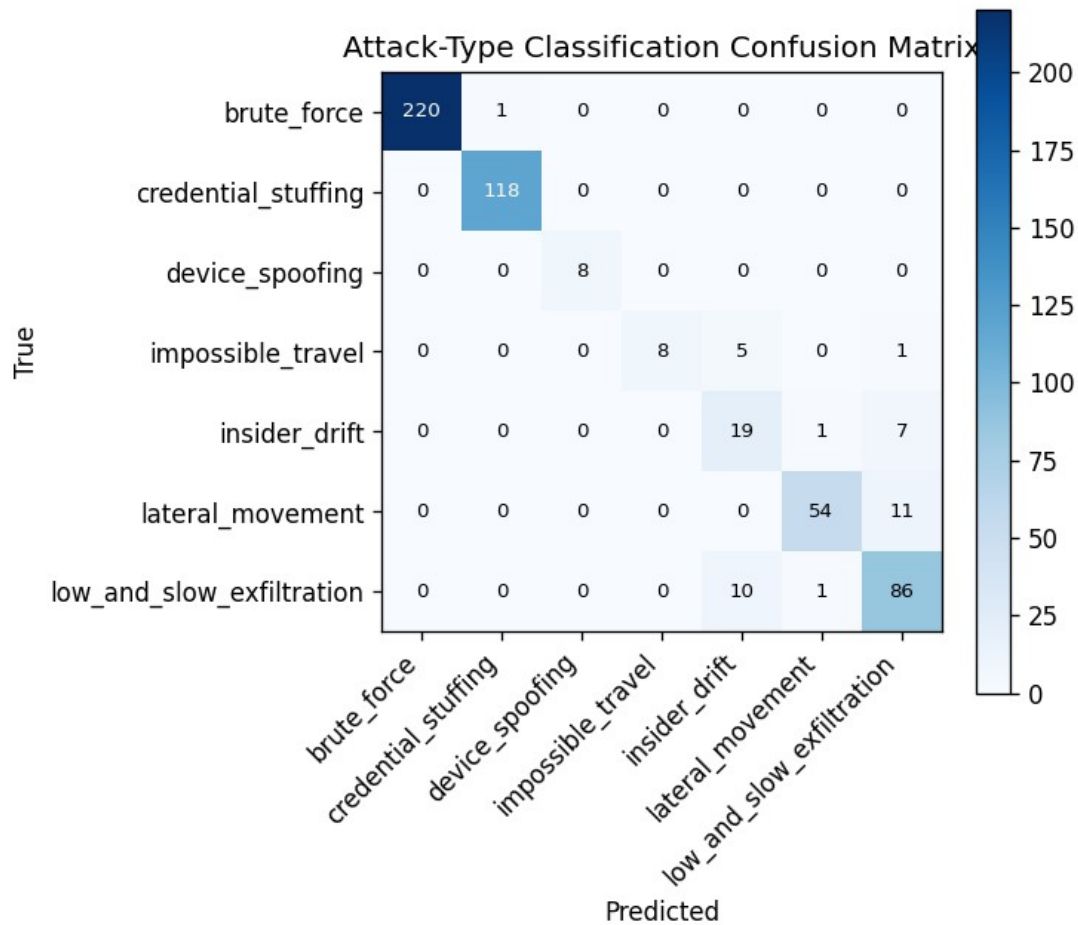
Risk Score Distribution



Alert Budget Curve

4.2 Attack-Type Classification (oracle setting - evaluated on true anomalies)

Attack Type	Precision	Recall	F1	Support
brute force	1.00	1.00	1.00	221
credential stuffing	0.99	1.00	1.00	118
device spoofing	1.00	1.00	1.00	8
impossible travel	1.00	0.57	0.73	14
insider drift	0.56	0.70	0.62	27
lateral movement	0.96	0.83	0.89	65
low and slow exfiltration	0.82	0.89	0.85	97
Overall accuracy			0.933	550



Attack-Type Confusion Matrix

4.3 Top Classifier Feature Importances

Feature	Importance
log_implied_speed_kmh	0.166
new_device_flag	0.116
log_time_gap_seconds	0.116
new_resource_flag	0.087
duration_zscore	0.083
log_prior_session_count	0.078
hour_zscore	0.076
log_ip_recent_distinct_entities	0.067

5. End-to-End Attack Coverage (Full Dataset Alert Queue)

Distinct from the held-out test metrics above: this measures, for the FULL dataset, how many true attack sessions of each type actually produced a visible alert an analyst could act on. This is a stricter, more operationally relevant check.

Attack Type	Injected	Flagged	Flag Rate	Correctly Typed
brute force	1163	1105	95.0%	1104
credential stuffing	379	379	100.0%	379
device spoofing	25	25	100.0%	25
impossible travel	46	24	52.2%	24
insider drift	88	1	1.1%	0
lateral movement	177	8	4.5%	5
low and slow exfiltration	308	12	3.9%	12

Honest finding: loud, structurally distinctive patterns (brute force, credential stuffing, device spoofing) are flagged at 95-100%. Quiet, gradual patterns (lateral movement, low-and-slow exfiltration, insider drift) are flagged far less often in practice - as low as 1.1% for insider drift, which was deliberately designed as an ambiguous edge case. This reflects a genuine, well-understood limitation of threshold-based unsupervised detection tuned to an overall base rate: subtler patterns don't clear the same bar as louder ones. See Section 7 for discussion.

6. Explainability

Every alert carries two independent explanation layers:

1. **Detection-level** (why flagged at all) - features ranked by standardized deviation from normal-row statistics.
2. **Classification-level** (why this specific attack type) - exact SHAP (`shap.TreeExplainer`) feature attributions from the trained Random Forest, computed per predicted class.

Both layers were spot-checked for coherence: for `impossible_travel` alerts, SHAP correctly identifies implied travel speed as the dominant factor, matching that feature's #1 ranking in the classifier's global feature importances (Section 4.3) - independent confirmation that the model learned genuine signal.

7. Known Limitations & Assumptions

- **Risk-score scaling** is min-max normalized relative to whatever batch is being scored; a live streaming deployment should pin this to a fixed reference range computed at training time rather than recomputing per call.
- **Attack-type flag-rate imbalance** (Section 5): subtler patterns are caught far less often than loud ones in the full alert queue. A per-pattern-type contamination/threshold tuning pass, traded deliberately against false-positive rate, is the natural next step.
- **Classification is evaluated in an oracle setting** (given the true anomalies) to isolate classifier quality from detector recall; an end-to-end number restricted to the detector's actual flagged set is a reasonable further check.
- **Synthetic data**: geo-coordinates jitter around 12 fixed reference cities rather than a full geocoding service; `command_sequence` uses a small templated action vocabulary per resource type. Both are adequate for the modeling task but not a substitute for real telemetry. Full assumptions in `generator/ASSUMPTIONS.md`.
- **Cold-start handling**: entities with fewer than 5 prior sessions in their rolling window fall back to peer-group (same `entity_type`) statistics rather than individual baselines. Verified: 1.5% of all sessions used this fallback with non-degenerate feature values.

8. Deliverables Checklist

#	Deliverable	Status
1	Synthetic data generator + documented taxonomy	Done - <code>generator/</code> , <code>generator/ASSUMPTIONS.md</code>
2	Baseline profiling model	Done - <code>ml/profiler.py</code> , <code>ml/ip_features.py</code>
3	Detection model (sequence-aware, flags deviations)	Done - <code>ml/detector.py</code> (Isolation Forest)
4	Anomaly classification (attack category)	Done - <code>ml/classifier.py</code> (Random Forest)
5	Explainability layer	Done - <code>ml/explain.py</code> (z-score + SHAP)
6	Analyst-facing dashboard	Done - <code>frontend/dashboard.html</code>
7	Report (this document)	Done - auto-generated from live model output

This report was generated automatically from live evaluation data and the current database state - every number above reflects the actual current model run, not a hand-typed snapshot.